



Software Factory en la Era de la IA

Procesos

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Sdlc

Software Development Life Cycle (SDLC)

El **SDLC** es el marco que define las fases, actividades y deliverables del proceso de desarrollo de software, desde la concepción de una idea hasta su mantenimiento continuo.

Definición

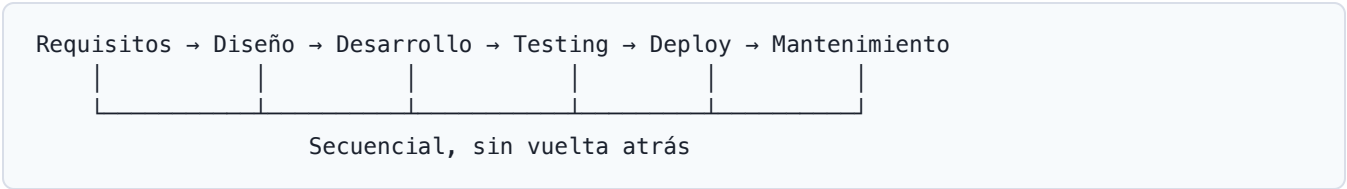
El Software Development Life Cycle es un proceso estructurado que guía el desarrollo de software desde los requisitos iniciales hasta el despliegue y mantenimiento, asegurando calidad, trazabilidad y eficiencia.

Fases del SDLC

Fase	Objetivo	Actividades principales	Deliverables
1. Planificación	Definir alcance y viabilidad	Análisis de negocio, estimación, riesgos	Business case, roadmap
2. Análisis	Capturar requisitos	Interviews, user stories, use cases	Requirements doc, backlog
3. Diseño	Arquitectura y diseño técnico	Diagramas, prototipos, ADRs	Design doc, architecture
4. Desarrollo	Implementar la solución	Coding, unit tests, code review	Código funcional
5. Testing	Verificar calidad	QA, UAT, performance, security	Test reports, bugs
6. Despliegue	Entregar a producción	CI/CD, release, rollback plan	Software desplegado
7. Mantenimiento	Soporte y evolución	Bug fixes, enhancements, monitoring	Patches, versiones

Modelos de SDLC

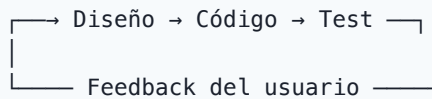
Waterfall (Cascada)



Ventaja	Desventaja
Simple de entender	Rígido, cambios costosos

Ventaja	Desventaja
Documentación completa	Feedback tardío
Fases claras	Largo tiempo-to-market

Iterativo



- Construcción en ciclos repetidos
- Cada iteración produce un incremento funcional
- Permite ajustar requisitos entre iteraciones

Modelo Espiral

Combina iterativo con gestión de riesgos:



- Cada ciclo incluye análisis de riesgo
- Ideal para proyectos grandes y complejos
- Permite prototipado temprano

Agile / Iterativo Ágil

El Manifiesto Ágil promueve:

Valor	Significado
Individuos > Procesos	Las personas y su interacción importan más
Software funcional > Documentación	Entregar valor, no paperwork
Colaboración con cliente > Contrato	Feedback continuo
Responder al cambio > Seguir un plan	Adaptabilidad

En la práctica se materializa en:

- Scrum: sprints, roles, eventos
- Kanban: flujo continuo, WIP limits
- Extreme Programming (XP): pair programming, TDD, continuous integration

Comparación de modelos

Aspecto	Waterfall	Iterativo	Espiral	Agile
Feedback	Al final	Cada iteración	Cada ciclo	Continuo
Cambios	Difíciles	Moderados	Controlados	Bienvenidos
Riesgo	Alto al final	Reducido	Gestionado activamente	Reducido
Documentación	Pesada	Moderada	Completa	Justa
Time-to-market	Largo	Moderado	Variable	Corto
Ideal para	Requisitos fijos	Proyectos medianos	Grandes y riesgosos	Evolución rápida

SDCI en la era de la IA

Con los LLMs y coding assistants, el SDLC evoluciona:

- **Planificación:** AI asiste en estimación y análisis de riesgos
- **Diseño:** Generación de arquitecturas y ADRs con AI
- **Desarrollo:** Copilot, code generation, autocomplete
- **Testing:** AI genera tests, detecta bugs, sugiere fixes
- **Despliegue:** Predictive deployment, anomaly detection
- **Mantenimiento:** AI para root cause analysis

Relación con otros conceptos

- Software Factory estandariza el SDLC
- DevOps automatiza las fases de deploy y operación
- Shift-Left mueve testing y seguridad hacia atrás
- Métricas miden la eficiencia del ciclo
- Herramientas soportan cada fase del SDLC

Kanban Software

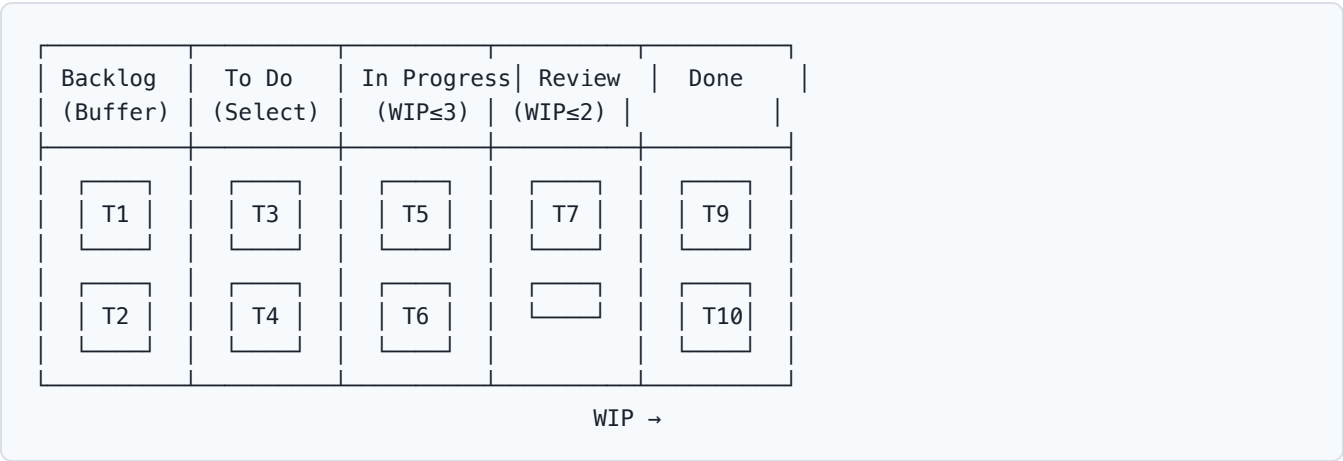
Kanban en Desarrollo de Software

Kanban es un método de gestión visual del trabajo que optimiza el flujo de valor mediante limitación del trabajo en progreso (WIP) y gestión pull. Originalmente de Toyota, adaptado al desarrollo de software.

Principios básicos de Kanban

Principio	Descripción
Comenzar con lo que hay	No reemplazar procesos existentes
Acordar aspiraciones de cambio	Mejora gradual, no revolución
Fomentar liderazgo en todos los niveles	Ownership distribuido
Gestionar el flujo	Enfocarse en el movimiento del trabajo
Hacer políticas explícitas	Reglas visibles en el tablero
Implementar feedback loops	Reuniones regulares, métricas
Mejorar experimentalmente	Hipótesis → experimento → resultado

El Tablero Kanban



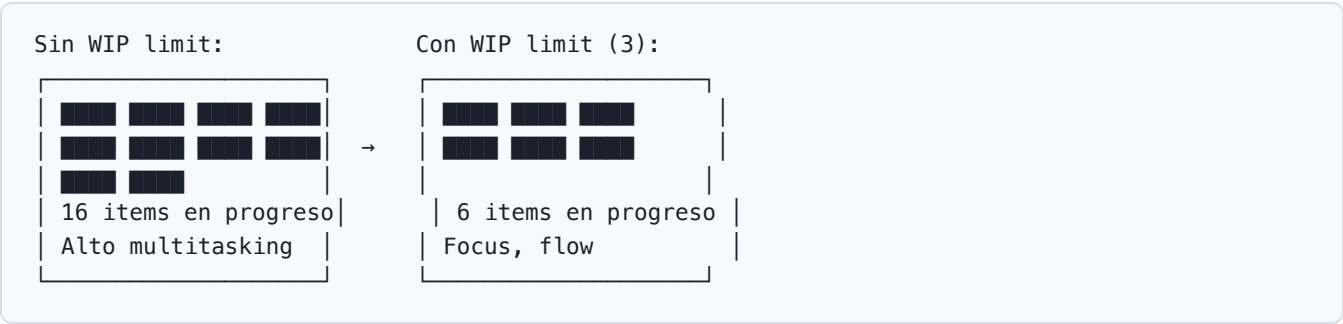
Columnas típicas

Columna	Propósito	WIP Limit
Backlog	Trabajo futuro priorizado	Sin límite
Ready / To Do	Seleccionado para el sprint/ciclo	Generalmente 1-2 x team size

Columna	Propósito	WIP Limit
In Progress	Trabajo activo	1-2 por persona
Review / QA	Esperando revisión o testing	2-4
Done	Completado y desplegado	Sin límite

WIP Limits (Límites de Trabajo en Progreso)

Los WIP limits son el **corazón de Kanban**:



Beneficios de WIP limits:

- Reducen context switching
- Exponen cuellos de botella
- Mejoran la predictibilidad
- Aceleran el throughput

Cómo establecer WIP limits:

1. Empezar generosamente (ej: team size $\times$ 1.5)
2. Observar el flujo por 2-4 semanas
3. Reducir gradualmente cuando el flujo sea estable
4. El WIP ideal es donde throughput $\times$ calidad se maximizan

Pull System (Sistema de Tirón)

El sistema de tirón es fundamental en Kanban:

Push System	Pull System
Se asigna trabajo	Se toma trabajo cuando hay capacidad
Sobrecarga frecuente	Flujo equilibrado
Prioridades cambiantes	Compromiso visual
Multitasking	Focus

Regla de oro: Nunca superar el WIP limit de una columna. Si está lleno, ayuda al upstream o haz review.

Cadencia y eventos de Kanban

Evento	Frecuencia	Propósito
Daily Kanban	Diaria (15 min)	Revisar flujo, impedimentos
Replenishment	Semanal	Seleccionar nuevo trabajo
Service Delivery Review	Quincenal/mensual	Evaluar métricas, mejorar
Kanban Retro	Mensual	Proceso y mejoras

Reunión diaria de Kanban

Enfocada en el **flujo**, no en el status:

- 1. ¿Hay tarjetas bloqueadas?
- 2. ¿Alguna columna supera WIP?
- 3. ¿Qué necesita ayuda urgente?
- 4. ¿El flujo está saludable?

Métricas de Kanban

Métrica	Fórmula	Objetivo
Lead Time	Finish date – Start date	Minimizar
Cycle Time	Done – In Progress start	Minimizar
Throughput	Items completados / período	Maximizar
WIP	Items activos	Mantener bajo
Flow Efficiency	Time active / total time	Maximizar
Age	Días desde creación	Alertar items viejos

Ver más en Métricas de Flujo.

Kanban vs Scrum

Aspecto	Kanban	[[../03-Procesos/03-scrum	Scrum]]
Cambio	Continuo	Solo entre sprints	
Roles	No definidos	SM, PO, Dev Team	
Eventos	Opcionales	Obligatorios	
WIP	Limitado por columna	Limitado por sprint	
Entrega	Continua	Al final del sprint	

Aspecto	Kanban	[[../03-Procesos/03-scrum	Scrum]]
Planificación	Pull on demand	Sprint planning	
Ideal para	Operaciones, soporte	Desarrollo de producto	

Implementación gradual

1. **Visualizar**: mapear el flujo actual en un tablero
2. **Limitar WIP**: establecer límites iniciales conservadores
3. **Gestionar flujo**: identificar y resolver cuellos de botella
4. **Implementar feedback loops**: reuniones periódicas
5. **Mejorar experimentalmente**: cambios pequeños y medidos

Relación con otros conceptos

- Lean Software es la base filosófica de Kanban
- Pull systems son el mecanismo central
- Métricas validan la salud del flujo
- Prácticas complementan con engineering practices
- DevOps integra Kanban en operaciones

Scrum

Scrum Framework

Scrum es el marco Agile más utilizado para gestionar trabajo complejo, enfocado en entregar valor incremental en iteraciones time-boxed llamadas **sprints**.

Definición

Scrum es un framework ligero que ayuda a personas, equipos y organizaciones a generar valor mediante soluciones adaptativas para problemas complejos.

Los 3 pilares de Scrum

Pilar	Significado	Práctica
Transparencia	Todo el trabajo es visible	Backlog visible, daily, dashboards
Inspección	Revisar frecuentemente	Sprint Review, retrospectives
Adaptación	Ajustar cuando sea necesario	Cambios en backlog, procesos

Los 5 valores de Scrum

1. **Courage** — Hacer lo correcto, enfrentar problemas
2. **Focus** — Concentrarse en el trabajo del sprint
3. **Respect** — Valorar las perspectivas de todos
4. **Openness** — Compartir progreso y desafíos
5. **Commitment** — Comprometerse con el sprint goal

Roles de Scrum

Product Owner (PO)

- Ver detalle en Estructura
- Dueño del Product Backlog
- Define prioridades y ROI
- Representa al usuario/cliente
- **No** gestiona personas, gestiona valor

Scrum Master (SM)

- Facilitador del proceso
- Elimina impedimentos
- Protector del equipo

- Coach en prácticas Agile
- **No** es manager, es servant leader

Development Team

- Cross-funcional (7±2 personas)
- Auto-organizado
- Comprometido con el sprint goal
- Responsable colectiva del deliverable

Artefactos de Scrum

Product Backlog



Campo	Descripción
User Story	Quién, qué, por qué
Acceptance Criteria	Cómo verificar completitud
Estimación	Story points o tiempo
Prioridad	Valor de negocio + riesgo

Sprint Backlog

Subconjunto del Product Backlog seleccionado para el sprint:

- Sprint Goal
- Selección de items del Product Backlog
- Plan de entrega

Increment

El resultado tangible al final del sprint:

- Debe ser "Done" (Definition of Done)

- Potencialmente desplegable
- Integra todo el trabajo del sprint

Eventos de Scrum (Ceremonias)

```

Sprint (2-4 semanas)
├─ Sprint Planning ( inicio)
│   ├── ¿Qué entregaremos?
│   └── ¿Cómo lo haremos?
├─ Daily Scrum (diario, 15 min)
│   ├── ¿Qué hice ayer?
│   ├── ¿Qué haré hoy?
│   └── ¿Hay impedimentos?
├─ Sprint Review (final)
│   ├── Demo del incremento
│   └── Feedback del stakeholder
└─ Sprint Retrospective (final)
    ├── ¿Qué salió bien?
    ├── ¿Qué mejorar?
    └── Acción concreta
  
```

Sprint Planning

Pregunta	Respuesta esperada
¿Qué podemos entregar?	Selección del backlog
¿Cómo lo lograremos?	Descomposición en tareas
¿Cuánto tiempo tenemos?	Time-box de 8h (sprint de 2 sem)

Daily Scrum

Formato eficiente (3 preguntas):

1. ¿Contribuí al sprint goal?
2. ¿Contribuiré al sprint goal hoy?
3. ¿Hay impedimentos?

Anti-patterns a evitar:

- Status report al manager
- Sesiones > 15 minutos
- Solo hablan 2-3 personas

Sprint Review

- Demo del incremento funcional
- Feedback de stakeholders

- Ajuste del Product Backlog
- **No** es una presentación de PowerPoint

Sprint Retrospective

Método	Formato
Start/Stop/Continue	Qué empezar, parar, continuar
Mad/Sad/Glad	Emociones del equipo
4Ls	Liked, Learned, Lacked, Longed for
Sailboat	Viento (ayuda), Anclas (frenan), Roca (riesgo)

Sprint

Aspecto	Recomendación
Duración	1-4 semanas (típico: 2 semanas)
Consistencia	Misma duración siempre
Buffer	No incluir vacaciones, feriados
Velocity	Se estabiliza tras 3-5 sprints

Definition of Done (DoD)

Checklist que todos los items deben cumplir:

- ☐ Código revisado y aprobado
- ☐ Tests unitarios pasando
- ☐ Tests de integración pasando
- ☐ Sin bugs críticos o altos abiertos
- ☐ Documentación actualizada
- ☐ Desplegado en ambiente de staging
- ☐ Aceptado por PO

Métricas de Scrum

Métrica	Descripción	Cómo usarla
Velocity	Story points completados/sprint	Planificación, no performance
Sprint Goal Rate	% de sprints que logran el goal	Compromiso y planificación
Defect Escape Rate	Bugs que llegan a producción	Calidad del proceso
Cycle Time	Tiempo medio de completar item	Eficiencia del flujo

Ver más en Métricas de Flujo.

Scrum vs Kanban

Aspecto	Scrum	[./03-Procesos/02-kanban-software	Kanban]]
Cambios	No en sprint	Continuos	
WIP	Por sprint	Por columna	
Roles	SM, PO, Team	Ninguno definido	
Events	Obligatorios	Opcionales	
Ideal para	Producto nuevo	Soporte, mantenimiento	

Relación con otros conceptos

- Manifiesto Ágil fundamenta Scrum
- Product Owner es un rol clave
- SFAFe/LeSS escalan Scrum a múltiples equipos
- Code Review complementa la calidad en el sprint
- Lean complementa con eficiencia

Scaled Agile Safe

Scaling Agile — SAFe, LeSS, Nexus

Cuando una organización crece más allá de un solo equipo Scrum, necesitan **marcos para escalar Agile**. Cada framework resuelve la coordination multi-equipo de forma diferente.

¿Cuándo escalar?

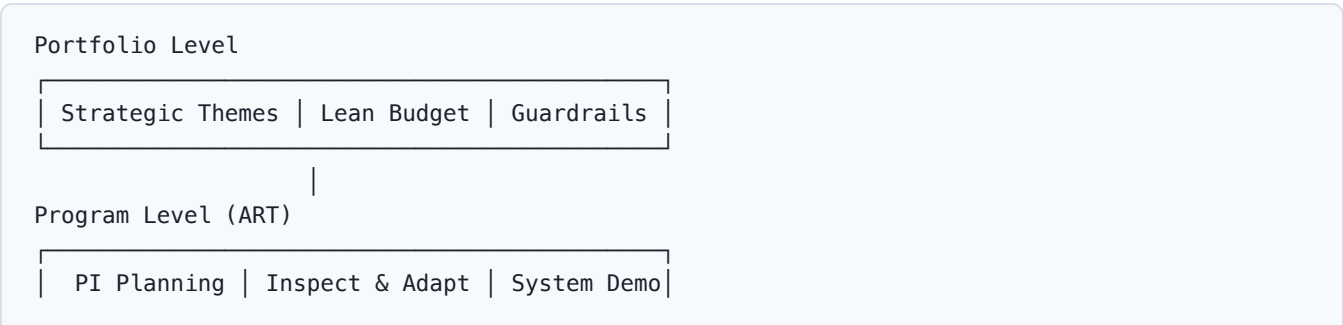
Señal	Descripción
Más de 1 equipo trabajando en el mismo producto	Coordinación necesaria
Dependencias frecuentes entre equipos	Bloqueos de entrega
Integración compleja de componentes	Riesgo de incompatibilidad
Roadmap compartido múltiples equipos	Sincronización de releases
Regulación y compliance	Gobernanza formal

Comparación general

Aspecto	SAFe	LeSS	Nexus	Spotify Model
Complejidad	Alta	Baja	Media	Media
Equipos	50-100+	2-8	3-9	Ilimitado
Ceremonias	Muy estructuradas	Mínimas	Leve	Flexibles
Roles nuevos	PI, ART, RTE	Ninguno	Integration Team	Tribe Lead
Framework	Pesado	Light	Leve	Cultural
Adopción	Top-down	Bottom-up	Bottom-up	Orgánica

SAFe (Scaled Agile Framework)

El framework más utilizado a gran escala:



Team 1	Team 2	Team 3	Team 4
Scrum	Scrum	Kanban	Scrum

Conceptos clave de SAFe

Concepto	Descripción
ART (Agile Release Train)	Grupo de equipos sincronizados
PI (Program Increment)	Iteración de 8-12 semanas (5 sprints)
PI Planning	Evento de planificación big-room
RTE (Release Train Engineer)	Facilitador del ART
System Demo	Demo integrada de todos los equipos
Inspect & Adapt	Retrospectiva de PI
Lean Budgets	Presupuesto por valor, no por proyecto

PI Planning (2 días)

Día 1:

- Visión del negocio y arquitectura
- Contexto de los equipos
- Identificación de dependencias

Día 2:

- Planificación por equipo
- Scrum of Scrums
- Presentación de riesgos
- Commitment del equipo

Ventajas y desventajas de SAFe

Ventajas	Desventajas
Estructura clara para grandes orgs	Pesado, mucha ceremonia
Alineamiento con negocio	Puede generar burocracia
Soporte de herramientas (Jira, Azure)	Requiere coaches certificados
Escalabilidad probada	Riesgo de "Agile en nombre solamente"

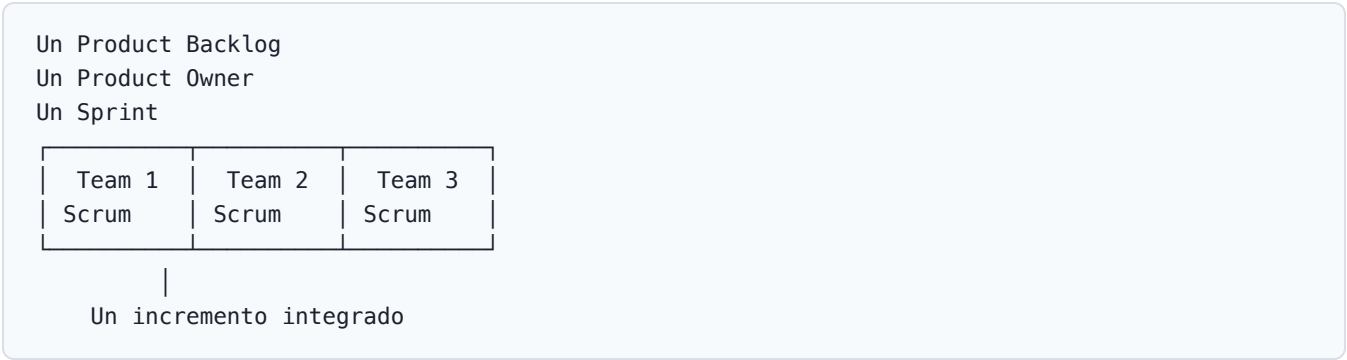
LeSS (Large-Scale Scrum)

Menos es más — extiende Scrum a múltiples equipos con cambios mínimos:

Principios de LeSS

- 1. **Deep empiricism** — transparencia, inspección, adaptación
- 2. **Whole product focus** — producto completo, no componentes
- 3. **Customer-centric** — valor para el usuario final
- 4. **Continuous improvement** — retrospectives en todos los niveles

Estructura LeSS



Regla clave: todos los equipos comparten el mismo Product Backlog y entregan el mismo incremento.

LeSS Huge

Para más de 8 equipos:

- **Area Requirements** — subdivisiones del backlog
- **Component Owners** — dueños de áreas específicas
- Mantiene la simplicidad de LeSS

Ventajas de LeSS

- Mínima ceremonia adicional
- No introduce roles nuevos
- Mantiene la esencia de Scrum
- Escalable sin complejidad

Nexus

Framework de Scrum.org para 3-9 equipos:

Elementos únicos de Nexus

Elemento	Descripción
Nexus Integration Team	Equipo dedicado a integración
Nexus Sprint Planning	Planificación cross-equipo

Elemento	Descripción
Nexus Daily Scrum	Coordinación diaria
Nexus Sprint Review	Demo integrada
Nexus Sprint Retrospective	Mejora cross-equipo

Artefactos adicionales

- **Nexus Sprint Backlog** — items que requieren trabajo cross-equipo
- **Integration Increment** — build integrado que todos los equipos producen

Spotify Model (no es un framework)

Referencia cultural más que proceso formal:

Squad (equipo自治) → Tribe (agrupación) → Chapter (especialidad) → Guild (comunidad)

Concepto	Descripción
Squad	Equipo cross-funcional auto-organizado
Tribe	Agrupación de squads relacionados
Chapter	Personas con misma habilidad en squads
Guild	Comunidad de práctica transversal

Ver más en Spotify Model.

¿Qué framework elegir?

Situación	Recomendación
2-8 equipos, mismo producto	LeSS
3-9 equipos, necesidad de integración	Nexus
10+ equipos, regulación, alineamiento con negocio	SAFe
Cultura fuerte, autonomía	Spotify Model
Equipos diversos, diferentes ritmos	Kanban a escala

Relación con otros conceptos

- Scrum es la base de todos estos frameworks
- Equipos Autónomos son la unidad fundamental
- Spotify Model es una referencia cultural
- Ley de Conway explica por qué la estructura importa

- Lean complementa con eficiencia

Ci Cd Pipeline

CI/CD Pipeline

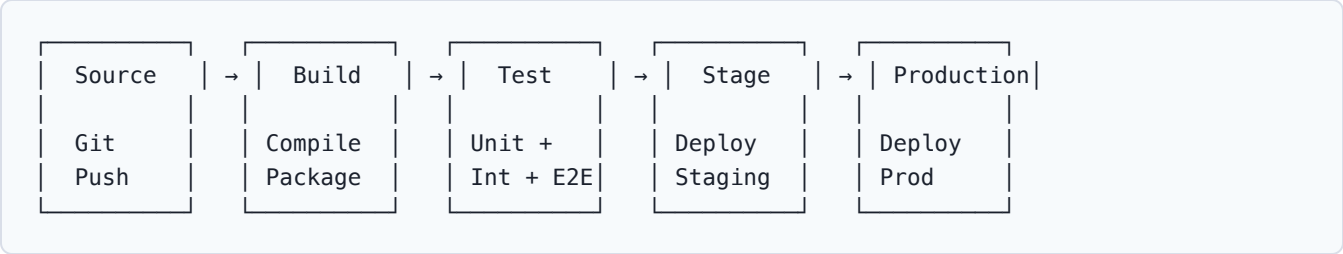
El **pipeline de CI/CD** es el corazón de la automatización en una Software Factory: integra, testea y despliega código de forma continua y confiable.

Definiciones

Término	Significado
CI (Continuous Integration)	Fusionar código frecuentemente con build y test automáticos
CD (Continuous Delivery)	CI + preparar código para deploy con un botón
CD (Continuous Deployment)	Delivery + desplegar automáticamente a producción

Continuous Integration → Continuous Delivery → Continuous Deployment
Build + Test + Staging Ready + Auto Deploy

Etapas del Pipeline



1. Source (Código fuente)

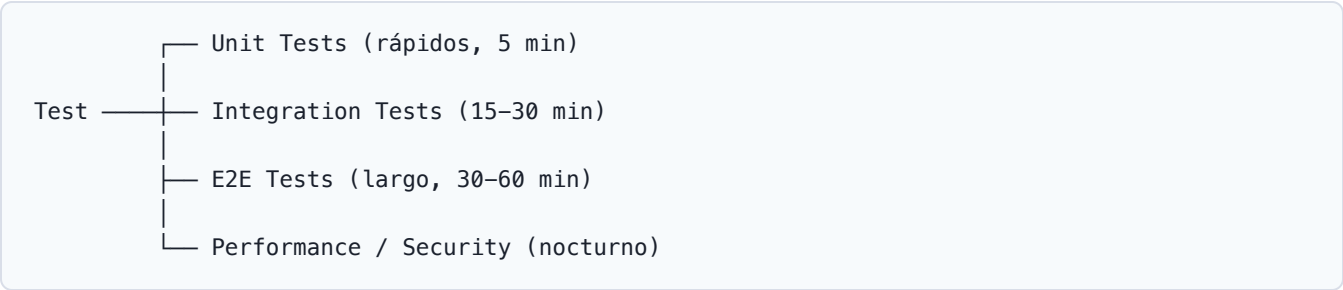
- Trigger: push a branch o PR
- Validación de branch naming
- Commit lint (conventional commits)

2. Build (Compilación)

Actividad	Herramienta
Compilación	Maven, Gradle, npm, Go build
Empaquetamiento	Docker, JAR, WAR, binary
Análisis estático	SonarQube, ESLint, RuboCop

Actividad	Herramienta
Snyk, Trivy	Vulnerability scan

3. Test (Pruebas)



Nivel	Cobertura objetivo	Tiempo
Unit	> 80%	< 5 min
Integration	> 60%	< 30 min
E2E	Flujos críticos	< 60 min
Performance	Benchmarks	Nocturno

4. Stage (Staging)

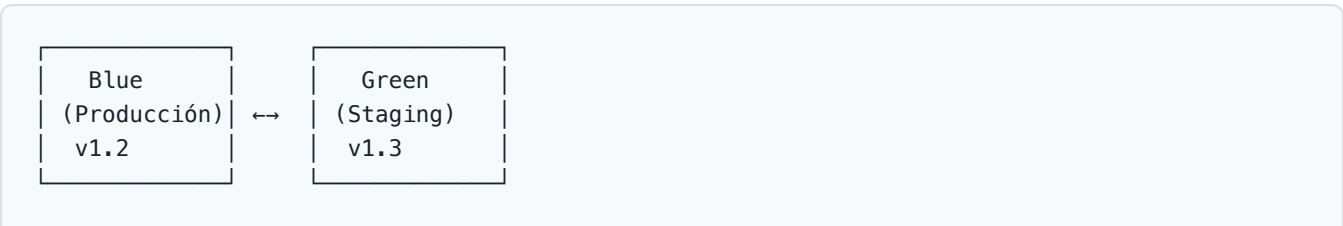
- Deploy a ambiente similar a producción
- Smoke tests automáticos
- Validación manual opcional
- Performance testing

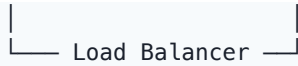
5. Production

- Deploy automatizado o con approval
- Health checks post-deploy
- Rollback automático si falla
- Monitoring y alertas activos

Estrategias de Deployment

Blue-Green Deployment





- Dos ambientes idénticos
- Swap instantáneo entre versiones
- Rollback = volver al anterior

Canary Deployment

Users → 95% → v1.2 (estable)
 └ 5% → v1.3 (nueva)

- Liberar a un % pequeño de usuarios
- Monitorear métricas clave
- Escalar gradualmente o revertir

Rolling Update

Step 1: [v1.2] [v1.2] [v1.2] [v1.2]
Step 2: [v1.3] [v1.2] [v1.2] [v1.2]
Step 3: [v1.3] [v1.3] [v1.2] [v1.2]
Step 4: [v1.3] [v1.3] [v1.3] [v1.2]
Step 5: [v1.3] [v1.3] [v1.3] [v1.3]

- Actualizar instancias una por una
- Sin downtime
- Rollback gradual

Feature Flags

```
if (featureFlags.isEnabled('new-checkout')) {  
  showNewCheckout();  
} else {  
  showLegacyCheckout();  
}
```

- Deploy código sin activar features
- Activar/desactivar por usuario, region, %
- Eliminar flags tras validación

Herramientas CI/CD

Categoría	Herramientas
CI/CD Servers	Jenkins, GitHub Actions, GitLab CI, CircleCI
Cloud Native	AWS CodePipeline, Azure DevOps, GCP Cloud Build
Containers	Docker, Kubernetes, Helm
IaC	Terraform, Ansible, Pulumi
Artifact Registry	Nexus, JFrog, Docker Hub, ECR
Secrets	Vault, AWS Secrets Manager, SOPS

Ver más en Herramientas.

Mejores prácticas

1. **Pipeline como código** — versionar el pipeline (Jenkinsfile, .gitlab-ci.yml)
2. **Build reproducible** — mismo input = mismo output siempre
3. **Fast feedback** — build < 10 min, tests < 30 min
4. **No flaky tests** — tests inestables se arreglan o eliminan
5. **Rollback automático** — si health check falla, revertir
6. **Secrets nunca en código** — usar vaults y masked variables
7. **Artifact immutable** — un build, un artifact, deploy el mismo artifact

Métricas de CI/CD

Métrica	Objetivo
Build Time	< 10 minutos
Lead Time	< 1 hora (commit → deploy)
Deployment Frequency	Diaria o más
Change Failure Rate	< 5%
MTTR	< 1 hora

Ver más en Métricas de Flujo.

Relación con otros conceptos

- DevOps se materializa en el CI/CD pipeline
- Shift-Left incluye testing y seguridad en el pipeline
- Trunk-Based Dev es la estrategia de branching ideal para CI/CD
- Release Management define cuándo y cómo deployar
- Prácticas complementan con engineering excellence

Trunk Based Development

Trunk-Based Development

Trunk-Based Development (TBD) es una estrategia de branching donde todos los desarrolladores trabajan en una rama principal (trunk/main) con branches de vida muy corta (horas, no días).

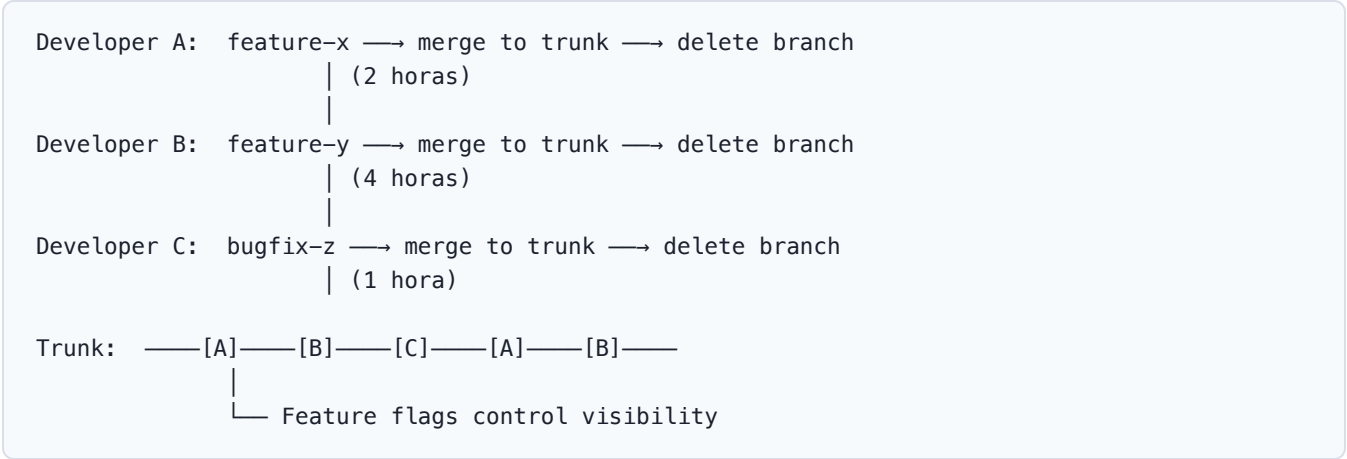
Definición

Trunk-Based Development es una práctica donde todo el código se integra en la rama principal al menos una vez al día, usando feature flags para controlar funcionalidad incompleta.

Reglas fundamentales

Regla	Descripción
Solo trunk	Todos trabajan en main o branches < 1 día
Integración diaria	Merge al trunk al menos 1 vez por día
Feature flags	Controlar features incompletas en producción
CI obligatorio	Build y tests deben pasar antes de merge
Code review rápido	PRs revieweados en < 1 hora

Flujo de trabajo



Branches de vida corta

Estrategia	Vida de branch	Ejemplo
Feature branch	1-5 días	feature/login-oauth

Estrategia	Vida de branch	Ejemplo
Short-lived	< 1 día	fix/null-pointer
Trunk-only	0 (direct push)	Solo con alta confianza

La regla de oro: si tu branch vive más de 1 día, necesitas split en tareas más pequeñas o usar feature flags.

Feature Flags

```
# Feature flag para nueva funcionalidad
if feature_flags.is_enabled('new-search', user=user):
    return new_search_algorithm(query)
else:
    return legacy_search(query)
```

Tipos de feature flags

Tipo	Duración	Ejemplo
Release flags	Días-semanas	Ocultar feature hasta release
Experiment flags	Semanas-meses	A/B testing
Ops flags	Permanente	Kill switch, rate limiting
Permission flags	Permanente	Feature tiers, beta access

Gestión de feature flags

- **Registry centralizado:** LaunchDarkly, Unleash, Flagsmith
- **Naming consistente:** team-feature-description
- **Cleanup post-release:** eliminar flags tras validación
- **Documentación:** cada flag tiene owner y fecha de expiración

TBD vs GitFlow

Aspecto	TBD	[[../03-Procesos/07-gitflow-alternativas	GitFlow]]
Branches	1 principal + cortas	Múltiples largas	
Integración	Diaria	Al final de feature	
Merge conflicts	Frecuentes pero pequeños	Pocos pero grandes	
Release	Continuo con flags	Por branch de release	
Rollback	Feature flag off	Revertir merge	
Velocidad	Más rápida	Más lenta	

Aspecto	TBD	[[../03-Procesos/07-gitflow-alternativas	GitFlow]]
Complejidad	Baja (branches)	Alta (múltiples branches)	

Beneficios

- 1. **Integración continua real** — no hay "merge hell"
- 2. **Feedback rápido** — código compartido al mismo día
- 3. **Menos conflictos** — cambios pequeños y frecuentes
- 4. **Deploy continuo** — siempre listo para producción
- 5. **Simplificación** — no hay branches de release, develop, etc.

Desafíos y mitigaciones

Desafío	Mitigación
Build trunk roto	CI obligatorio, tests rápidos
Features incompletas	Feature flags
Code review rápido	SLA de < 1 hora para reviews
Tests lentos	Parallelización, test splitting
Testing en trunk	Flaky test elimination

Requisitos para TBD exitoso

Trunk-Based Development

- └ CI pipeline rápido (< 10 min)
- └ Tests estables y rápidos
- └ Feature flags management
- └ Code review culture (< 1 hr)
- └ Monitoring y alertas
- └ Rollback capability

Métricas de éxito

Métrica	Objetivo
Commit frequency	> 1 por persona/día
Branch lifetime	< 1 día
Integration frequency	> 1 por persona/día
Build break rate	< 2%
PR review time	< 1 hora

Adopción gradual

1. **Empezar con bugfixes** — fixes van directo a trunk
2. **Features cortas** — branches de < 1 día
3. **Feature flags** — para features más largas
4. **Eliminar branches largas** — split en tareas más pequeñas
5. **Deploy continuo** — trunk siempre deployable

Relación con otros conceptos

- GitFlow es la alternativa tradicional
- CI/CD es el habilitador técnico de TBD
- Code Review debe ser rápido para TBD
- Release Management usa feature flags
- DevOps promueve integración continua

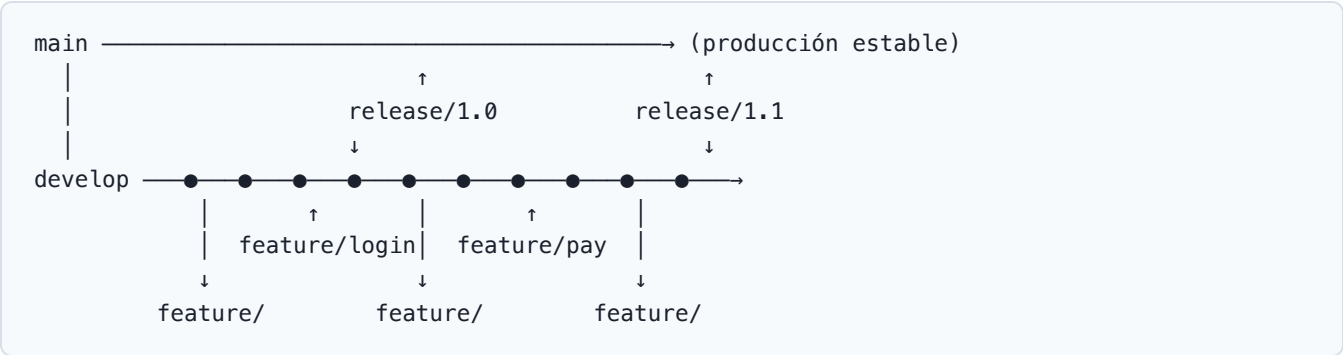
Gitflow Alternativas

GitFlow y Alternativas

Las **branching strategies** definen cómo los equipos organizan el código en ramas, cuándo crearlas, fusionarlas y eliminarlas. Elegir la estrategia correcta impacta directamente la velocidad y calidad de entrega.

GitFlow

Creada por Vincent Driessen en 2010, es la estrategia más estructurada:



Ramas en GitFlow

Rama	Propósito	Vida
main	Código desplegado en producción	Permanente
develop	Última funcionalidad para release	Permanente
feature/	Nuevas funcionalidades	Semanas
release/	Preparación de release	Días-semanas
hotfix/	Fixes urgentes en producción	Horas

Flujo de GitFlow

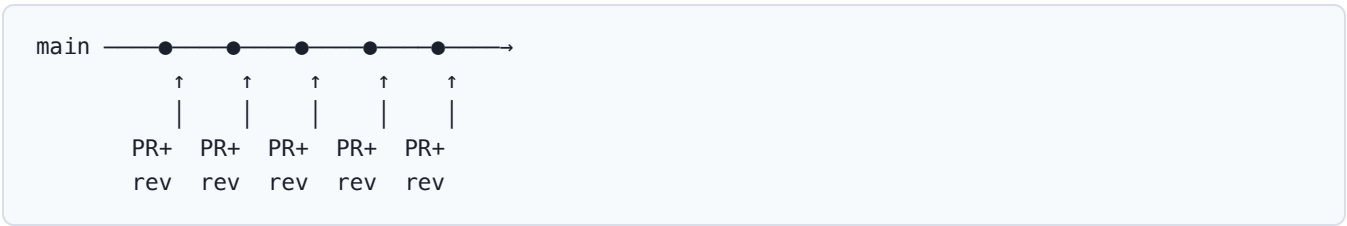
1. Crear rama develop desde main
2. Crear feature/xxx desde develop
3. Merge feature → develop (PR + review)
4. Crear release/1.0 desde develop
5. Testing, bug fixes en release
6. Merge release → main + develop
7. Tag en main: v1.0
8. Hotfix: branch desde main, merge a main + develop

Cuándo usar GitFlow

Situación	¿GitFlow?
Release con versión definida	Sí
Múltiples versiones en paralelo	Sí
Deploy continuo	No
CI/CD maduro	No
Equipo pequeño	No

GitHub Flow

Estrategia simple y popular en GitHub:



Reglas de GitHub Flow

1. **main** siempre deployable
2. Crear branch descriptiva desde main
3. Commits frecuentes con push
4. Abrir PR para discusión y review
5. Merge después de approval
6. Deploy desde main

Ventajas

- Simple, fácil de entender
- Ideal para deploy continuo
- Menos merge conflicts
- Perfecto para TBD

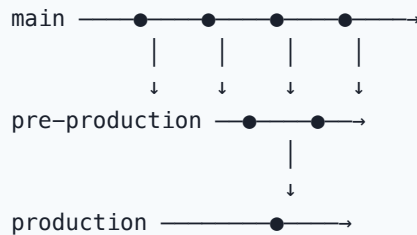
Cuándo usar GitHub Flow

Situación	¿GitHub Flow?
Deploy continuo	Sí
SaaS, web apps	Sí
Equipo distribuido	Sí

Situación	¿GitHub Flow?
Release con versiones	Limitado
Hotfixes frecuentes	Sí

GitLab Flow

Variante que agrega ramas de environment:



Ramas de environment

Rama	Propósito
main	Código más reciente
pre-production	Staging/QA
production	Producción

Release branches en GitLab Flow



Cuándo usar GitLab Flow

Situación	¿GitLab Flow?
Deploy continuo	Sí
Múltiples ambientes	Sí
Release branches	Sí
Infraestructura como código	Sí

Trunk-Based Development

Ver artículo completo.

Resumen rápido:

- Solo trunk (main) + branches de < 1 día
- Feature flags para control
- Deploy continuo
- La estrategia más recomendada para CI/CD moderno

Comparación de estrategias

Aspecto	GitFlow	GitHub Flow	GitLab Flow	TBD
Complejidad	Alta	Baja	Media	Baja
Deploy	Por release	Continuo	Continuo	Continuo
Release branches	Sí	No	Sí	No
Feature flags	No	No	Opcional	Sí
Hotfix	Rama especial	Branch normal	Branch normal	Directo a trunk
Ideal para	Versionado	SaaS	Multi-env	CI/CD maduro

Criterios de decisión

```
¿Deploy continuo?  
├─ Sí → ¿Feature flags disponibles?  
│   ├── Sí → Trunk-Based Development  
│   └─ No → GitHub Flow  
└─ No → ¿Release con versiones?  
    ├── Sí → ¿Múltiples versiones en paralelo?  
    │   ├── Sí → GitFlow  
    │   └─ No → GitLab Flow  
    └─ No → GitHub Flow
```

Anti-patterns comunes

Anti-pattern	Problema	Solución
Branches largos	Merge hell, feedback lento	TBD o GitHub Flow
Sin PR review	Código sin calidad	Code review obligatorio
Merge sin test	Build roto	CI obligatorio
Feature flags forever	Deuda técnica	Cleanup post-release
Desarrollar en main	Build roto frecuentemente	Branches cortas

Relación con otros conceptos

- TBD es la tendencia moderna
- CI/CD habilita estrategias simples
- Code Review es independiente de la estrategia
- Release Management define la cadencia
- Herramientas soportan estas estrategias

Code Review Proceso

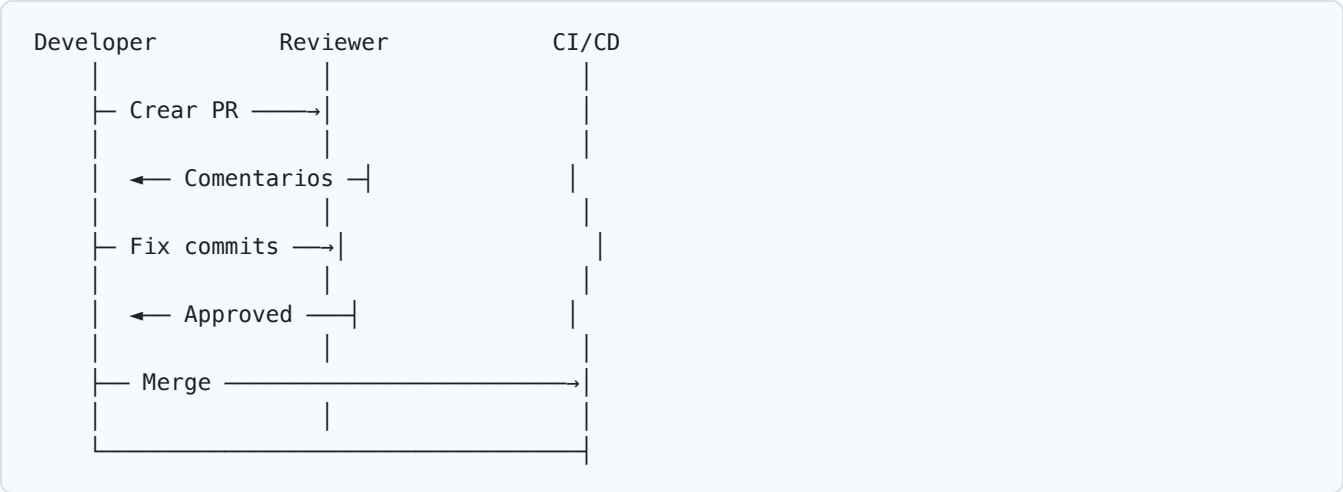
Proceso de Code Review

El **Code Review** es el proceso donde otros desarrolladores revisan el código antes de integrarlo, buscando calidad, corrección y mejora compartida de conocimiento.

¿Por qué hacer Code Review?

Beneficio	Descripción
Calidad	Detectar bugs antes de llegar a producción
Conocimiento	Compartir contexto y aprendizaje
Consistencia	Mantener estándares de código
Seguridad	Identificar vulnerabilidades
Mantenibilidad	Código más legible y comprensible

Flujo del Code Review



Paso a paso

1. **Autor crea PR** con descripción clara
2. **CI ejecuta** tests y quality checks
3. **Reviewer asignado** revisa en < 1 hora
4. **Feedback inline** con sugerencias concretas
5. **Autor resuelve** comentarios
6. **Approved** y merge automático
7. **Branch eliminada** tras merge

El autor del PR

Antes de abrir el PR

- ☐ Código compila sin errores
- ☐ Tests pasan localmente
- ☐ No hay TODOs sin resolver
- ☐ Branch está actualizada con main

Descripción del PR

```
## What
- Describir qué cambia

## Why
- Razón del cambio (link a issue/ticket)

## How
- Approach técnico elegido

## Testing
- Qué tests se agregaron/modificaron

## Screenshots
- Si hay cambio visual
```

Respondiendo a feedback

- **No tomar personal** — el objetivo es mejorar el código
- **Explicar decisiones** — por qué se eligió ese approach
- **Aceptar sugerencias** — cuando tiene sentido
- **Discutir con argumentos** — no con autoridad

El reviewer

Checklists de review

Funcionalidad

- ☐ ¿El código hace lo que se propone?
- ☐ ¿Los edge cases están manejados?
- ☐ ¿La lógica es correcta?
- ☐ ¿Los tests cubren los escenarios principales?

Código

- ☐ ¿Es legible y comprensible?

- ☐ ¿Sigue los principios SOLID?
- ☐ ¿Nombres descriptivos y consistentes?
- ☐ ¿No hay duplicación significativa?
- ☐ ¿Manejo de errores apropiado?

Seguridad

- ☐ ¿Hay inputs sin sanitizar?
- ☐ ¿Secrets hardcodeados?
- ☐ ¿SQL injection, XSS, CSRF?
- ☐ ¿Autenticación y autorización correctas?

Performance

- ☐ ¿Queries N+1?
- ☐ ¿Memoria suficiente?
- ☐ ¿Locking adecuado?
- ☐ ¿Algoritmos eficientes?

Mantenibilidad

- ☐ ¿Se puede entender sin contexto extra?
- ☐ ¿Documentación actualizada?
- ☐ ¿Feature flags para features nuevas?
- ☐ ¿Backward compatible?

Métricas de Code Review

Métrica	Objetivo	Fórmula
Review Time	< 1 hora	Merge time – PR creation
Review Depth	> 4 comentarios	Comentarios / reviewer
Approval Rate	> 80% first pass	Approved / total PRs
Defect Density	< 1 bug / 100 LOC	Bugs found / LOC reviewed

SLAs recomendados

Tamaño PR	Tiempo máximo de review
< 100 LOC	30 minutos
100-400 LOC	1 hora
400-1000 LOC	2 horas
> 1000 LOC	Rechazar — dividir en PRs más pequeños

Herramientas de automatización

Herramienta	Propósito
ESLint / RuboCop	Linting automático
SonarQube	Quality gate, bugs, code smells
Snyk / Trivy	Vulnerability scanning
Codecov	Cobertura de código
Danger.js	Checks customizados en PR
GitHub CODEOWNERS	Auto-asignación de reviewers

Anti-patterns

Anti-pattern	Problema	Solución
Rubber stamping	Aprobar sin revisar	Checklists, SLAs
Nitpicking excesivo	PRs retrasados	Priorizar bugs > style
Review masivo	PRs de > 1000 LOC	Split en PRs pequeños
Solo 1 reviewer	Knowledge silo	Mínimo 2 reviewers
Review post-merge	Sin valor real	Review pre-merge obligatorio

Code Review en la práctica

Pair Review (para código crítico)

- 2 personas revisan juntas en tiempo real
- Ideal para: seguridad, arquitectura, cambios complejos
- Herramientas: VS Code Live Share, Tuple

Async Review (para código routine)

- Revisión asíncrona vía PR
- Ideal para: features, bug fixes, refactors
- SLA: < 1 hora para PRs pequeños

Relación con otros conceptos

- TBD requiere reviews rápidos
- CI/CD ejecuta checks automáticos
- SOLID guía qué buscar en review
- Prácticas complementan con pair programming

- Métricas miden la efectividad del proceso

Release Management

Release Management

Release Management es el proceso de planificar, coordinar y ejecutar la entrega de software a producción de forma controlada y predecible.

Definición

Release Management abarca las actividades necesarias para planificar, programar, construir, probar y desplegar releases de software de manera que se maximice el valor y se minimice el riesgo.

Cadencia de releases

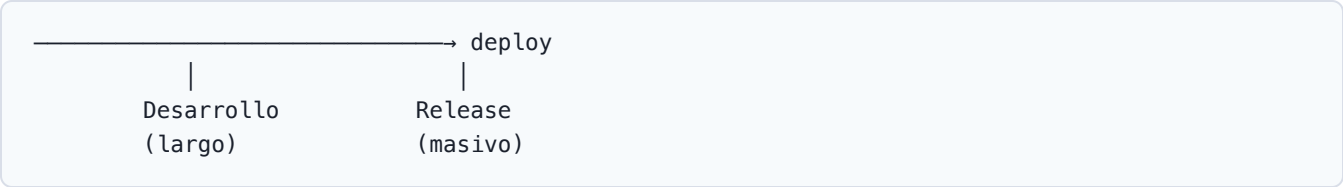
Cadencia	Descripción	Ejemplo
On-demand	Cuando hay valor para entregar	Continuous deployment
Time-based	Frecuencia fija predecible	Semanal, quincenal, mensual
Milestone-based	Al completar features clave	Por epic o feature set
Event-based	Alineado con eventos externos	Conferencia, Black Friday

Calendario de releases

Ene	Feb	Mar	Abr
— R24.1 —	— R24.2 —	— R24.3 —	— R24.4 —
Sprint 1-2	Sprint 3-4	Sprint 5-6	Sprint 7-8
Feature A	Feature B	Feature C	Feature D

Estrategias de release

Big Bang Release



- Todo se despliega de una vez
- Alto riesgo, alta coordinación
- Ideal para: v1.0, cambios radicales

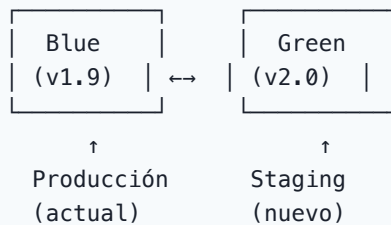
- **Riesgo:** si falla, todo falla

Rolling Release

Users: → gradualmente todos migrados
 Region 1: v2.0 ✓
 Region 2: v2.0 ✓
 Region 3: v1.9 → v2.0
 Region 4: v1.9

- Despliegue gradual por regiones/grupos
- Reduce impacto de fallos
- Ideal para: servicios globales
- **Requiere:** feature flags, monitoring

Blue-Green Deployment



- Swap instantáneo entre versiones
- Rollback en segundos
- **Requiere:** 2 ambientes idénticos

Canary Release

100% → gradualmente a 100%
 |
 5% → usuarios seleccionados prueban v2.0
 |
 | Métricas OK → escalar
 | Métricas NO → revert

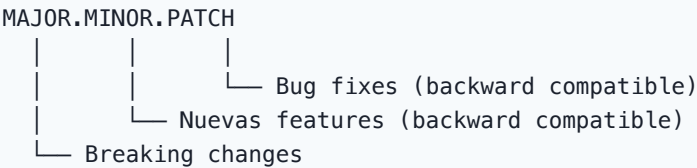
- Liberar a % pequeño de usuarios
- Monitorear métricas clave
- Ideal para: validación en producción
- **Requiere:** feature flags, observabilidad

Feature Flags (Sin release tradicional)

Deploy = código en producción
Release = activar feature flag

- Código siempre desplegado
- Activación/desactivación remota
- A/B testing integrado
- Rollback instantáneo

Versionado semantic (SemVer)



Ejemplos

Versión	Cambio
1.0.0 → 1.0.1	Bug fix
1.0.1 → 1.1.0	Nueva feature
1.1.0 → 2.0.0	Breaking change

Changelog automático

```
# Changelog

## [2.1.0] - 2024-03-15
### Added
- Feature X con Y capacidades
- Soporte para Z

### Fixed
- Bug en módulo A
- Performance issue en B

### Changed
- API de C actualizada (breaking)
```

Proceso de release

1. Planificación → ¿Qué va en esta release?
2. Feature Freeze → No más features nuevas
3. Code Freeze → Solo bug fixes
4. QA / UAT → Validación final
5. Release Notes → Documentar cambios
6. Deploy → Despliegue a producción
7. Monitoring → Observar métricas
8. Rollback Plan → Si algo falla
9. Post-Release → Retrospectiva

Release Notes template

```
## Release X.Y.Z – [Fecha]

### 🚀 Nuevas Features
– Feature A: descripción (link a docs)

### 🐛 Bug Fixes
– Fix B: descripción (link a issue)

### ⚠️ Breaking Changes
– API C: [descripción + migración]

### 📦 Deprecations
– Feature D: será eliminada en v3.0

### 🔗 Links
– Documentación: [link]
– Migración: [link]
```

Métricas de releases

Métrica	Objetivo
Release Frequency	Al menos quincenal
Release Size	< 500 LOC por release
Rollback Rate	< 5%
Release Lead Time	< 1 semana (plan → deploy)
Post-release Defects	< 2 por release

Relación con otros conceptos

- CI/CD automatiza el pipeline de release
- TBD habilita releases continuos

- Incidentes pueden ocurrir post-release
- Change Management controla el proceso
- Métricas miden la efectividad

Incident Management

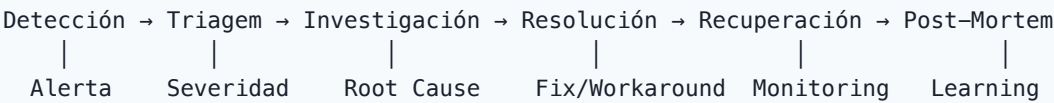
Gestión de Incidentes

La **Gestión de Incidentes** es el proceso de detectar, responder, resolver y aprender de interrupciones en servicios de producción, minimizando impacto al negocio y usuarios.

Definición

Incident Management es el conjunto de procesos para identificar, analizar y resolver anomalías en sistemas de producción, restaurando el servicio lo más rápido posible y documentando el aprendizaje.

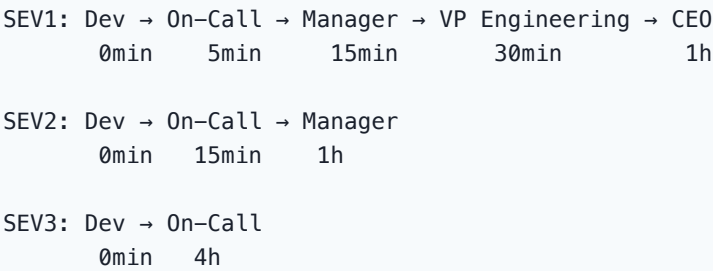
Ciclo de vida de un incidente



Niveles de severidad

Severidad	Impacto	Tiempo de respuesta	Ejemplo
SEV1 (Crítico)	Servicio caído, pérdida de datos	5 minutos	API principal down, data breach
SEV2 (Mayor)	Degradación significativa	15 minutos	Performance degradada, feature rota
SEV3 (Menor)	Impacto limitado	1 hora	Bug en función no crítica
SEV4 (Bajo)	Sin impacto al usuario	Siguiente día	Error en logs, cosmetic bug

Escalamiento automático



On-Call (Garantía)

Estructura de rotación

Aspecto	Recomendación
Duración del turno	1 semana (no más)
Horario	24/7 con compensación
Handoff	Documentación al cambiar turno
Compensación	Pago extra, tiempo libre

On-Call Engineer checklist

```
## Inicio de turno
- [ ] Revisar alertas activas
- [ ] Verificar dashboards
- [ ] Confirmar acceso a herramientas
- [ ] Revisar incidentes abiertos

## Durante el turno
- [ ] Responder alertas según SLA
- [ ] Documentar acciones en runbook
- [ ] Escalar si es necesario

## Fin de turno
- [ ] Documentar incidentes resueltos
- [ ] Handoff al siguiente on-call
- [ ] Resumen de estado actual
```

Reglas para on-call saludable

1. **Máximo 1 semana** por rotación
2. **Compensación justa** (dinero o tiempo libre)
3. **Runbooks actualizados** para cada alerta
4. **Escalamiento claro** — nunca solo
5. **Post-shift recovery** — día libre tras semana de on-call intenso

Runbooks

Documentos paso-a-paso para resolver incidentes conocidos:

```
# Runbook: API High Latency

## Síntomas
- Alerta: p99 latency > 2s
- Users report slowness

## Diagnóstico
1. Verificar dashboard de latency
2. Revisar CPU/memory de pods
```

3. Verificar DB connection pool

Acción

1. Si DB pool saturated:
kubecttl scale deployment api --replicas=6
2. Si CPU high:
kubecttl top pods -l app=api
Escalar horizontalmente

Verificación

- Latency vuelve a < 500ms
- Error rate < 0.1%

Escalamiento

- Si no se resuelve en 15min: SEV2
- Contactar: @oncall-backend

Tipos de runbooks

Tipo	Propósito	Ejemplo
Response runbook	Resolver incidente específico	"API down"
Escalation runbook	Cuándo y cómo escalar	"SEV1 procedure"
Recovery runbook	Restaurar servicio	"DB failover"
Diagnostic runbook	Investigar problema	"High latency checklist"

Post-Mortem (Análisis post-incidente)

Estructura del post-mortem

Post-Mortem: [Incidente]

Resumen

- ****Fecha****: YYYY-MM-DD
- ****Duración****: X horas Y minutos
- ****Severidad****: SEV1/2/3
- ****Impacto****: X usuarios afectados, \$Y perdidos

Timeline

- 14:00 - Alerta detectada
- 14:05 - On-call notificado
- 14:10 - Investigación inicia
- 14:25 - Root cause identificado
- 14:35 - Fix implementado
- 14:45 - Servicio restaurado

Root Cause

```
- [Descripción técnica del problema]

## Contributing Factors
- Factor 1: [descripción]
- Factor 2: [descripción]

## What Went Well
- [Lista de cosas que funcionaron]

## What Could Improve
- [Lista de mejoras]

## Action Items
| Acción | Owner | Fecha | Estado |
|-----|-----|-----|-----|
| [Acción 1] | @dev1 | 2024-04-01 | Pendiente |
| [Acción 2] | @dev2 | 2024-04-15 | Pendiente |
```

Blameless Post-Mortem

- Principios fundamentales:
- **Enfocarse en el sistema**, no en la persona
 - **Qué salió mal**, no quién falló
 - **Mejorar procesos**, no castigar
 - **Sicología segura** para reportar errores

"Las personas no fallan, los sistemas fallan."

Herramientas de gestión de incidentes

Categoría	Herramientas
Alerting	PagerDuty, Opsgenie, Grafana Alerting
Monitoring	Prometheus, Datadog, New Relic
Communication	Slack (#incident), Statuspage
Tracking	Jira (incidents), PagerDuty incidents
Post-mortem	Confluence, Notion, GitHub

Métricas de incidentes

Métrica	Objetivo
MTTD (Mean Time To Detect)	< 5 minutos
MTTA (Mean Time To Acknowledge)	< 5 minutos

Métrica	Objetivo
MTTR (Mean Time To Resolve)	< 1 hora (SEV1)
Incident Frequency	< 2 SEV1/mes
Escalation Rate	< 30%

Ver más en Métricas de Operaciones.

Incident Response checklist

```
## Durante el incidente
- [ ] Crear canal de comunicación (#incident-YYYY-MM-DD)
- [ ] Asignar Incident Commander
- [ ] Documentar timeline en tiempo real
- [ ] Comunicar a stakeholders cada 30 min
- [ ] Implementar fix o workaround

## Después del incidente
- [ ] Confirmar resolución
- [ ] Comunicar resolución
- [ ] Programar post-mortem (dentro de 48h)
- [ ] Crear action items en Jira
- [ ] Actualizar runbooks
```

Relación con otros conceptos

- Change Management previene incidentes
- Problem Management investiga causas raíz
- DevOps promueve blameless culture
- CI/CD reduce incidentes con calidad
- Métricas miden la efectividad de respuesta

Change Management

Change Management

Change Management es el proceso formal de evaluar, aprobar e implementar cambios en sistemas de producción, asegurando que se realicen de manera segura y controlada.

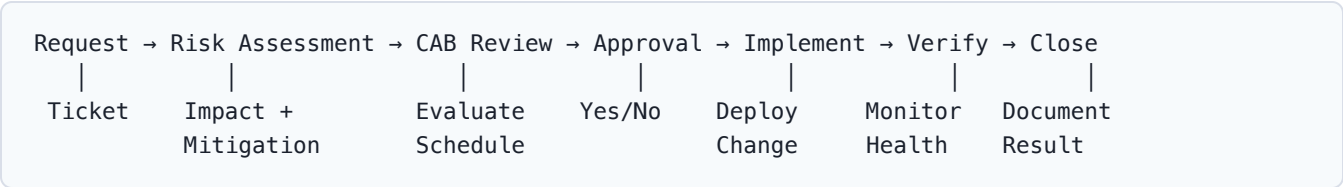
Definición

Change Management es el proceso que controla los cambios en la infraestructura y servicios de TI, minimizando el riesgo de interrupciones y maximizando la estabilidad del servicio.

Tipos de cambios

Tipo	Riesgo	Proceso	Ejemplo
Standard	Bajo	Pre-aprobado	Patch de seguridad conocido
Normal	Medio	CAB review	Nueva feature, upgrade de librería
Emergency	Alto	Expedited CAB	Hotfix de seguridad, fix de producción

Proceso de Change Management



Paso 1: Change Request

```
# Change Request: CR-2024-0123

## Resumen
- Upgrade de PostgreSQL de 14 a 15

## Justificación
- CVE de seguridad en v14
- Soporte fino en EOL

## Alcance
- Base de datos principal (producción)
- 3 replicas de read

## Riesgos identificados
```

- Breaking changes en queries
- Downtime de 5-10 minutos

Mitigaciones

- Testing en staging completo
- Backup antes del cambio
- Rollback plan documentado

Ventana de cambio

- Fecha: 2024-03-15 02:00-04:00 UTC
- Impacto esperado: < 10 minutos downtime

Paso 2: Risk Assessment

Factor de riesgo	Puntuación	Descripción
Impacto al negocio	1-5	1=bajo, 5=crítico
Probabilidad de fallo	1-5	1=improbable, 5=seguro
Complejidad	1-5	1=simple, 5=complejo
Reversibilidad	1-5	1=fácil revertir, 5=irreversible
Ventana de cambio	1-5	1=flexible, 5=urgente

Score total:

- 1-8: Standard change (pre-aprobado)
- 9-16: Normal change (CAB review)
- 17-25: High risk (CAB + management approval)

Paso 3: CAB (Change Advisory Board)

Rol	Responsabilidad
CAB Chair	Facilita la reunión, toma decisión
Change Manager	Evalúa completitud de la solicitud
Technical Lead	Evalúa viabilidad técnica
Operations	Evalúa impacto en infraestructura
Security	Evalúa riesgos de seguridad
Business	Evalúa impacto en negocio

Reunión del CAB

Agenda típica (30 min):

- 1. Revisar cambios propuestos (5 min cada uno)
- 2. Preguntas y discusión
- 3. Decisión: Aprobar / Rechazar / Diferir
- 4. Asignar ventana de cambio

Paso 4: Implementación

```
## Pre-deploy
- [ ] Notificar a stakeholders
- [ ] Backup completo
- [ ] Verificar staging
- [ ] Confirmar rollback plan

## Deploy
- [ ] Ejecutar change según procedimiento
- [ ] Monitorear métricas en tiempo real
- [ ] Verificar health checks

## Post-deploy
- [ ] Confirmar servicios operacionales
- [ ] Monitorear por 30 minutos
- [ ] Documentar resultado
- [ ] Cerrar change request
```

Paso 5: Rollback Strategy

Estrategia	Cuándo usarla
Revert inmediato	Cambio simple, datos no modificados
Restore from backup	Cambio con datos, backup disponible
Feature flag off	Cambio detrás de feature flag
Blue-green swap	Ambiente dual disponible
Manual rollback	Procedimiento complejo documentado

```
## Rollback Plan: CR-2024-0123

### Trigger para rollback
- Error rate > 1% después de 5 minutos
- Latencia p99 > 2 segundos
- Health check falla

### Pasos de rollback
1. Detener migración
2. Restaurar backup de PostgreSQL 14
3. Reiniciar servicios
```

- 4. Verificar conectividad
- 5. Notificar a stakeholders

Tiempo estimado de rollback: 10 minutos

Data loss risk: Ninguno (backup pre-migración)

Change Management en DevOps

La filosofía DevOps transforma el Change Management:

Tradicional	DevOps
CAB manual para cada cambio	Auto-aprobación para cambios de bajo riesgo
Ventanas de cambio fijas	Deploy continuo
Documentación pesada	Pipeline como evidencia
Fear of change	Cambios pequeños y frecuentes

Automatización del Change Management

Pipeline automático:

- 1. Code review → Aprobación técnica
- 2. CI tests → Evidencia de calidad
- 3. Security scan → Aprobación de seguridad
- 4. Deploy approval → Aprobación de release
- 5. Monitoring → Evidencia de estabilidad

Métricas de Change Management

Métrica	Objetivo
Change Success Rate	> 95%
Change Failure Rate	< 5%
Lead Time	< 1 semana
CAB Approval Time	< 24 horas
Rollback Rate	< 3%

Compliance y auditoría

Requisito	Cómo cumplir
Trazabilidad	IDs de change en commits y deploys
Aprobación	Logs de CAB y approvals

Requisito	Cómo cumplir
Testing	Reportes de CI/CD como evidencia
Rollback	Plan documentado y probado
Post-change review	Post-mortems para cambios fallidos

Relación con otros conceptos

- Incidentes pueden requerir cambios emergency
- Release Management coordina los cambios
- Gobernanza Técnica define políticas
- DevOps automatiza el proceso
- CI/CD ejecuta los cambios

Problema Management

Problem Management

Problem Management es el proceso de identificar, analizar y eliminar la causa raíz de incidentes recurrentes, evitando que se repitan y reduciendo el impacto a largo plazo.

Definición

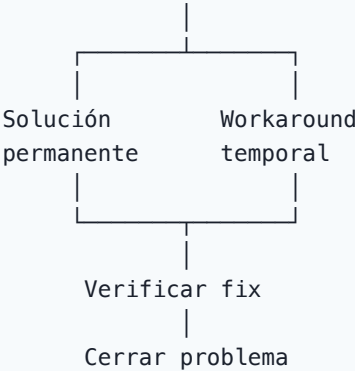
Problem Management es el proceso que gestiona la causa raíz de incidentes, tanto de forma proactiva (prevención) como reactiva (después de un incidente), para minimizar el impacto al negocio.

Problem Management vs Incident Management

Aspecto	[[../03-Procesos/10-incident-management	Incident Management]]	Problem Management
Objetivo	Restaurar servicio	Eliminar causa raíz	
Urgencia	Inmediata	Puede ser planificada	
Enfoque	Síntomas	Causas	
Duración	Minutos/horas	Días/semanas	
Resultado	Workaround o fix	Prevención permanente	

Flujo de Problem Management

Incidentes recurrentes → Identificar problema → Investigar causa raíz



Técnicas de Root Cause Analysis (RCA)

5 Whys (5 Porqués)

Técnica simple para llegar a la causa raíz:

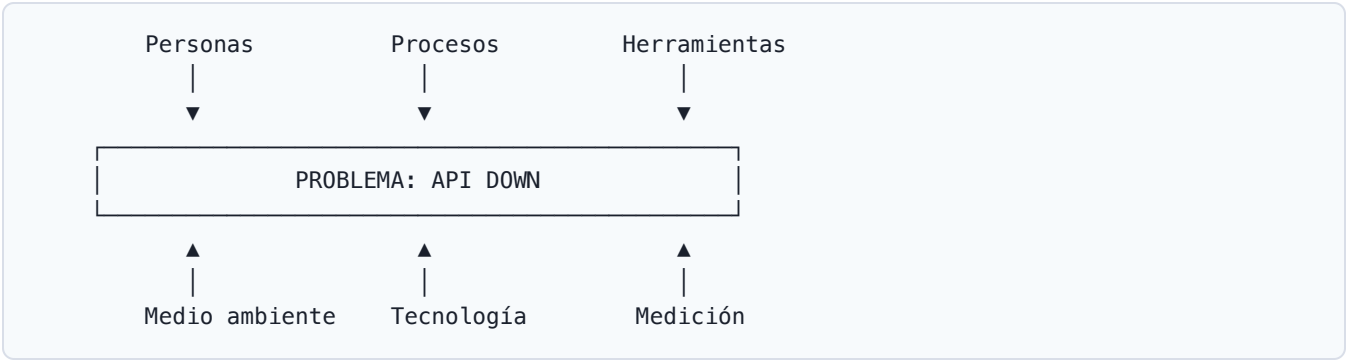
Problema: La API devolvió error 500

- ¿Por qué? → La base de datos no respondió
- ¿Por qué? → Se agotó el connection pool
- ¿Por qué? → Queries lentas consumían conexiones
- ¿Por qué? → Falta índice en tabla de transacciones
- ¿Por qué? → No se realizó capacity planning

Causa raíz: Falta de capacity planning + falta de índices
Solución: Añadir índices + alertas de connection pool

Nivel	Pregunta	Respuesta
1	¿Por qué falló?	DB no respondió
2	¿Por qué la DB no respondió?	Connection pool agotado
3	¿Por qué se agotó?	Queries lentas
4	¿Por qué eran lentas?	Falta índice
5	¿Por qué faltaba el índice?	No había proceso de revisión

Fishbone Diagram (Ishikawa)



Categorías típicas (6M):

Categoría	Ejemplos
Man (Personas)	Falta de capacitación, error humano
Machine (Tecnología)	Bug, hardware defectuoso
Material (Datos)	Datos corruptos, input inválido
Method (Procesos)	Proceso inadecuado, falta de testing
Measurement (Medición)	Métricas incorrectas, alertas falsas
Mother Nature (Entorno)	Cambio en infra, proveedor

5 Whys + Fishbone (combinado)

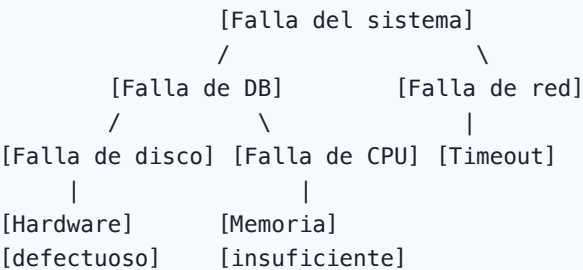
- 1. Hacer Fishbone para identificar categorías
- 2. Aplicar 5 Whys en cada categoría relevante
- 3. Priorizar causas por impacto
- 4. Crear action items por causa raíz

Pareto Analysis (80/20)

Causa	Frecuencia	% Acumulado
Falta de testing	15	35%
Config incorrecta	10	58%
Falta de monitoreo	8	77%
Bug en dependencia	5	89%
Error humano	3	96%

- Enfocarse en el 20% de causas que causan el 80% de problemas

Fault Tree Analysis (FTA)



Proceso completo de RCA

```
## Root Cause Analysis: Incident INC-2024-0456

### 1. Definición del problema
La API de pagos devolvió errores 500 durante 45 minutos el 15/03/2024.

### 2. Timeline
- 14:00 – Primera alerta de error 500
- 14:05 – Escalado a SEV2
- 14:15 – DB connection pool agotado identificado
- 14:30 – Workaround: reiniciar pods
- 14:45 – Servicio restaurado

### 3. Análisis de causa raíz (5 Whys)
```

[Ver análisis 5 Whys arriba]

4. Causas contribuyentes

- Falta de índice en tabla de transacciones
- Sin alertas de connection pool
- Capacity planning insuficiente

5. Acciones correctivas

Acción	Tipo	Owner	Fecha
-----	-----	-----	-----
Añadir índice DB	Correctiva	@dba	20/03
Alertas de connection pool	Preventiva	@sre	22/03
Capacity planning review	Preventiva	@architect	30/03
Load testing automatizado	Preventiva	@qa	15/04

6. Verificación

- Monitorear por 30 días
- Verificar que no hay incidentes similares
- Actualizar runbooks

Problem Management proactivo

Actividad	Descripción
Trend Analysis	Analizar incidentes para encontrar patrones
Capacity Planning	Predecir necesidades futuras
Maturity Assessment	Evaluar madurez del sistema
Technology Refresh	Actualizar componentes obsoletos
Architectural Review	Identificar fragilidades

Métricas de Problem Management

Métrica	Objetivo
Problems Identified	Tracking de problemas encontrados
Mean Time to Identify	< 7 días (proactivo)
Solutions Implemented	> 80% de action items completados
Incident Recurrence	< 10% de problemas se repiten
Known Errors	Base de datos de errores documentados

Knowledge Management

Artefacto	Propósito
Known Error Database	Errores conocidos y workarounds
Problem Records	Documentación completa por problema
Root Cause Database	Patrones de causa raíz recurrentes
Runbooks	Procedimientos de workaround

Relación con otros conceptos

- Incident Management provee datos para Problem Management
- Technical Debt es una causa frecuente de problemas
- Change Management implementa las soluciones
- Métricas habilitan el análisis de tendencias
- Lean promueve la eliminación de desperdicio

Value Stream Mapping

Value Stream Mapping en Software

Value Stream Mapping (VSM) es una herramienta de Lean que visualiza el flujo de valor desde la solicitud del cliente hasta la entrega, identificando desperdicio y oportunidades de mejora.

Definición

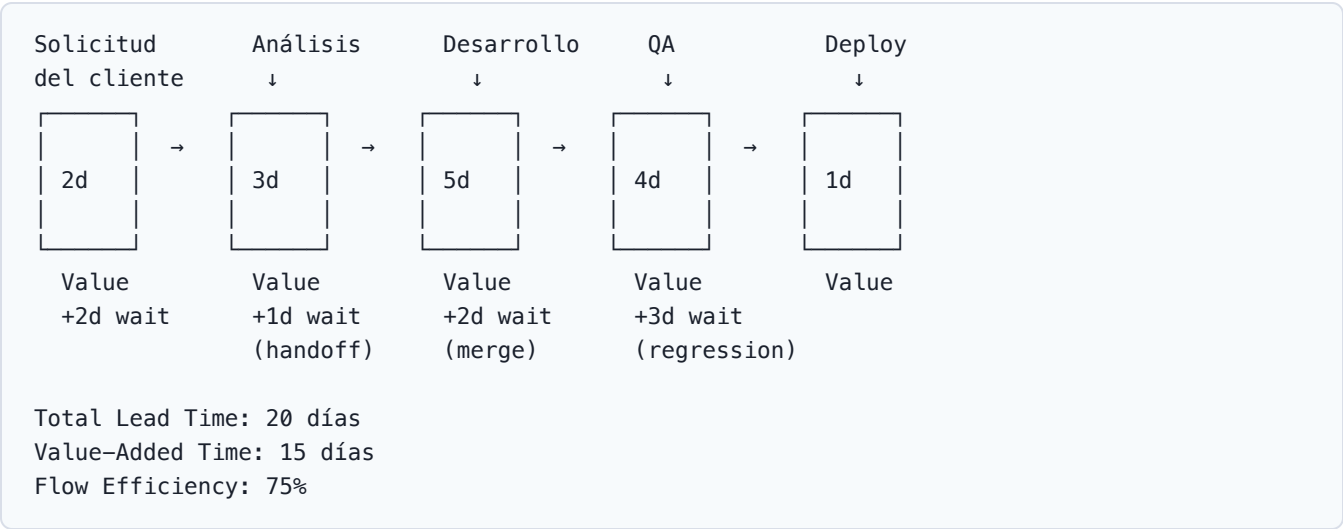
Value Stream Mapping es una técnica para visualizar y analizar el flujo de trabajo y materiales/información necesarios para entregar un producto o servicio al cliente, identificando valor agregado vs desperdicio.

¿Por qué VSM en Software?

Problema	Cómo VSM lo resuelve
Lead time largo	Identifica wait times y handoffs
Silos entre equipos	Visualiza el flujo completo
Ineficiencias ocultas	Mide tiempo real vs tiempo de trabajo
Mejoras dispersas	Enfoca en el cuello de botella más grande
Falta de visibilidad	Crea un mapa compartido del proceso

Mapa de Value Stream para Software

Estado actual (Current State)



Métricas en el mapa

Métrica	Cálculo	Ejemplo
Lead Time	Tiempo total de inicio a fin	20 días
Process Time	Tiempo de trabajo real	15 días
Wait Time	Tiempo de espera entre pasos	5 días
Flow Efficiency	Process Time / Lead Time	75%
Touch Time	Tiempo de interacción humana	10 días
Handoffs	Número de transferencias	4

Pasos para crear un VSM

Paso 1: Definir el alcance

¿Qué flujo vamos a mapear?

- Feature development (idea → producción)
- Bug fix (reporte → resuelto)
- Onboarding (solicitud → acceso)
- Incident (alerta → resuelto)

Paso 2: Recopilar datos

Dato	Cómo obtenerlo
Lead time	Git logs, Jira cycle time
Process time	Estimación del equipo
Wait time	Análisis de transiciones
Handoffs	Entrevistas al equipo
Batch size	Revisar PRs promedio
WIP	Contar items activos

Paso 3: Dibujar el mapa actual

FLUJO DE VALOR					
BACKLOG	DEV	REVIEW	QA	DEPLOY	
PT: 0 WT: 2d	PT: 3d WT: 1d	PT: 0.5d WT: 1d	PT: 2d WT: 3d	PT: 0.5d WT: 0d	

| Total LT: 10d | Value-Added: 6d | Efficiency: 60% |

PT = Process Time, WT = Wait Time

Paso 4: Identificar desperdicios (7 wastes de Lean)

Desperdicio	En Software	Ejemplo
Transport	Handoffs innecesarios	Dev → QA → Dev → QA
Inventory	WIP alto	20 items en progreso
Motion	Cambios de contexto	Multi-proyecto simultáneo
Waiting	Tiempo muerto	Esperando review, aprobación
Over-processing	Trabajo innecesario	Docs que nadie lee
Over-production	Features no usadas	Construir sin validar
Defects	Bugs y rework	Fix que genera otro bug

Paso 5: Diseñar el mapa futuro (Future State)

Estado Futuro: Eliminar wait times, reducir batch size

FLUJO DE VALOR			
BACKLOG	DEV+QA	REVIEW	DEPLOY (auto)
PT: 0	PT: 4d	PT: 0.5d	PT: 0.5d
WT: 0.5d	WT: 0.5d	WT: 0.5d	WT: 0d
Total LT: 6d Value-Added: 5d Efficiency: 83%			

- Mejoras:
- QA integrado (elimina wait de 3d)
 - Deploy automatizado
 - Pull system con WIP limits

Métricas de Value Stream

Métrica	Estado Actual	Estado Futuro	Objetivo
Lead Time	10 días	6 días	-40%

Métrica	Estado Actual	Estado Futuro	Objetivo
Process Time	6 días	5 días	-17%
Wait Time	4 días	1.5 días	-63%
Flow Efficiency	60%	83%	+23pp
Handoffs	4	2	-50%
Batch Size	5 PRs/sprint	2 PRs/sprint	-60%

Ver más en Métricas de Flujo.

Herramientas de VSM

Herramienta	Tipo
Miro / Mural	Colaborativo visual
Draw.io	Diagramas gratuitos
ValueStreamMap.com	Template online
Excel/Sheets	Datos y cálculos
Git + Jira	Datos de lead time automatizados

Value Stream Mapping en practice

Workshop de VSM (2-4 horas)

1. **Preparación** (antes): recopilar datos de lead time
2. **Dibujar mapa actual** (60 min): equipo completo
3. **Identificar desperdicios** (30 min): los 7 wastes
4. **Diseñar mapa futuro** (60 min): ideal state
5. **Crear plan de acción** (30 min): priorizar mejoras

Anti-patterns

Anti-pattern	Problema
Mapa perfecto sin datos	Sin evidencia, sin mejoras
Solo proceso, no personas	Ignora la realidad humana
Mapa estático	Nunca se actualiza
Enfoque en局部	Optimizar un paso no optimiza el flujo

Relación con otros conceptos

- Lean Software es la base filosófica de VSM
- Pull systems son el mecanismo de mejora
- Métricas de Flujo proveen los datos
- Kanban es la herramienta de implementación
- SDLC es el proceso que se mapea

Flujo Tirón Pull

Sistema de Tirón (Pull System) en Desarrollo

Un **sistema de tirón** (pull system) es un mecanismo donde el trabajo se mueve hacia adelante solo cuando hay capacidad en el siguiente paso, en contraste con el sistema de empuje (push) donde el trabajo se asigna independientemente de la capacidad.

Definición

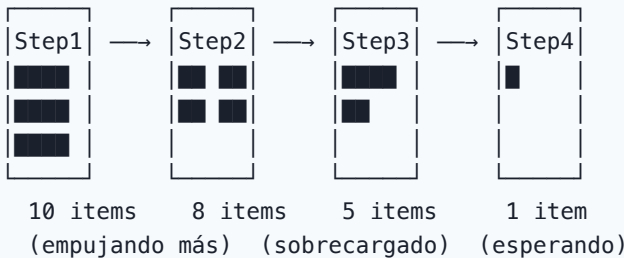
Un sistema de tirón es un enfoque de producción donde cada paso del proceso "tira" del trabajo del paso anterior solo cuando tiene capacidad disponible, regulando el flujo y evitando sobrecarga.

Push vs Pull

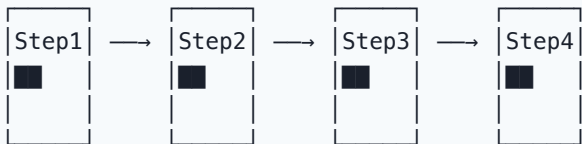
Aspecto	Push System	Pull System
Trabajo	Se empuja al siguiente paso	Se tira cuando hay capacidad
Sobrecarga	Frecuente	Minimizada
WIP	Ilimitado o alto	Controlado por límites
Lead time	Impredecible	Más predecible
Multitasking	Alto	Bajo
Flujo	Irregular	Equilibrado

Ejemplo visual

PUSH SYSTEM:



PULL SYSTEM:



2 items (WIP=2) 2 items (WIP=2) 2 items (WIP=2) 2 items (WIP=2)

WIP Limits (Límites de Trabajo en Progreso)

Los WIP limits son el **mecanismo central** del pull system:

Backlog → [Ready] → [In Progress] → [Review] → [Done]
WIP=5 WIP=3 WIP=2

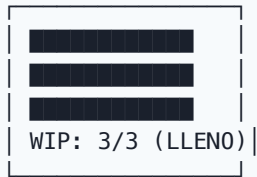
Cómo funcionan

Estado normal:



Puede agregar
más trabajo

WIP limit alcanzado:



NO puede agregar
más trabajo → HELP

Efectos de WIP limits

WIP Alto	WIP Bajo
Más multitasking	Focus en 1-2 items
Menos context switching	Más context switching
Lead time impredecible	Lead time predecible
Menos collaboration	Más collaboration
Más throughput individual	Menos throughput individual
Menos calidad	Más calidad

Calibración de WIP limits

1. **Empezar conservador**: team size × 1.5
2. **Observar 2-4 semanas**: medir throughput
3. **Reducir gradualmente**: si throughput estable
4. **Punto óptimo**: donde throughput × calidad se maximizan

Regla de Little's Law:

WIP = Throughput × Lead Time

Si quieres reducir Lead Time, reduce WIP.

Flow Efficiency (Eficiencia de Flujo)

Total Lead Time: 10 días
Value-Added Time: 3 días (trabajo real)
Wait Time: 7 días (esperando)

Flow Efficiency = 3/10 = 30%

Mejorar Flow Efficiency

Estrategia	Impacto
Reducir WIP	Menos esperas
Cross-training	Menos dependencias
Automatizar handoffs	Menos transferencias
Reducir batch size	Menos espera por batch
Integrar QA en dev	Eliminar cola de QA

Métricas de flujo

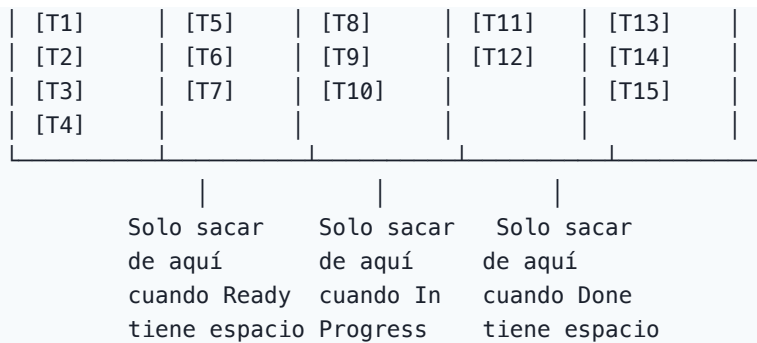
Métrica	Fórmula	Objetivo
Flow Efficiency	Value-Added Time / Total Time	> 50%
Lead Time	Finish – Start	Minimizar
Cycle Time	Done – In Progress start	Minimizar
Throughput	Items completados / período	Maximizar
WIP	Items activos	Mantener bajo
Age	Días desde creación	Alertar items viejos

Ver más en Métricas de Flujo.

Pull en la práctica

Kanban Board como Pull System

Backlog	Ready (WIP: 5)	In Progress (WIP: 3)	Review (WIP: 2)	Done
---------	-------------------	-------------------------	--------------------	------



Pull en CI/CD

Commit → Build → Test → Stage → Deploy
(WIP=5) (WIP=3) (WIP=2) (WIP=1)

Solo avanza cuando el siguiente paso tiene capacidad

Pull en Code Review

PR abierto → Reviewer disponible → Review completado
(WIP: 2 PRs max)

Anti-patterns de Pull System

Anti-pattern	Problema	Solución
WIP always at limit	WIP limit muy bajo	Aumentar o mejorar flujo
Bypass WIP limits	Sin disciplina	Enforcement en herramientas
No age alerts	Items estancados	Age-based escalation
Ignoring blocked items	Flujo bloqueado	Daily flow review
Push disguised as pull	"Solo esta vez"	Políticas explícitas

Implementación gradual

- 1. **Visualizar**: crear tablero con el flujo actual
- 2. **Medir**: establish baseline de lead time y throughput
- 3. **Limitar WIP**: empezar generosamente, reducir gradualmente
- 4. **Gestionar flujo**: identificar y resolver cuellos de botella
- 5. **Mejorar**: optimizar continuamente con datos

Relación con otros conceptos

- Kanban es la implementación práctica de pull
- Lean es la base filosófica
- VSM identifica dónde aplicar pull
- Métricas de Flujo miden la salud del pull system
- SDLC es el proceso que se optimiza

README

Procesos

Procesos, flujos de trabajo y marcos de gestión que organizan la entrega de software en una Software Factory.

Contenido

#	Archivo	Tema	Nivel	
01	[[../03-Procesos/01-sdlc	Software Development Life Cycle]]	Fases, modelos: waterfall, agile, spiral	Principiante
02	[[../03-Procesos/02-kanban-software	Kanban en Desarrollo]]	Principios, WIP limits, pull system, métricas	Intermedio
03	[[../03-Procesos/03-scrum	Scrum Framework]]	Roles, eventos, artefactos, sprint	Principiante
04	[[../03-Procesos/04-scaled-agile-safe	Scaling Agile (SAFe, LeSS, Nexus)]]	Marcos para escalar Agile	Avanzado
05	[[../03-Procesos/05-ci-cd-pipeline	CI/CD Pipeline]]	Etapas, herramientas, estrategias de deploy	Intermedio
06	[[../03-Procesos/06-trunk-based-development	Trunk-Based Development]]	vs GitFlow, feature flags, branches cortas	Intermedio
07	[[../03-Procesos/07-gitflow-alternativas	GitFlow y Alternativas]]	GitHub Flow, GitLab Flow, trunk-based	Intermedio
08	[[../03-Procesos/08-code-review-proceso	Proceso de Code Review]]	Flujo, checklist, automatización, métricas	Intermedio
09	[[../03-Procesos/09-release-management	Release Management]]	Cadencia, big bang, blue-green, canary	Avanzado
10	[[../03-Procesos/10-incident-management	Gestión de Incidentes]]	Severidad, on-call, runbooks, post-mortems	Intermedio
11	[[../03-Procesos/11-change-management	Change Management]]	CAB, risk assessment, rollback	Avanzado
12	[[../03-Procesos/12-problema-management	Problem Management]]	RCA, 5 Whys, fishbone, causa raíz	Avanzado
13	[[../03-Procesos/13-value-stream-mapping	Value Stream Mapping]]	Mapeo, optimización, métricas de flujo	Avanzado

#	Archivo	Tema	Nivel	
14	[[../03-Procesos/14-flujo-tirón-pull	Sistema de Tirón (Pull)]]	Pull systems, WIP, eficiencia de flujo	Intermedio

Secciones relacionadas

- 01 — Fundamentos
- 02 — Estructura
- 04 — Prácticas
- 05 — Herramientas
- 06 — Métricas